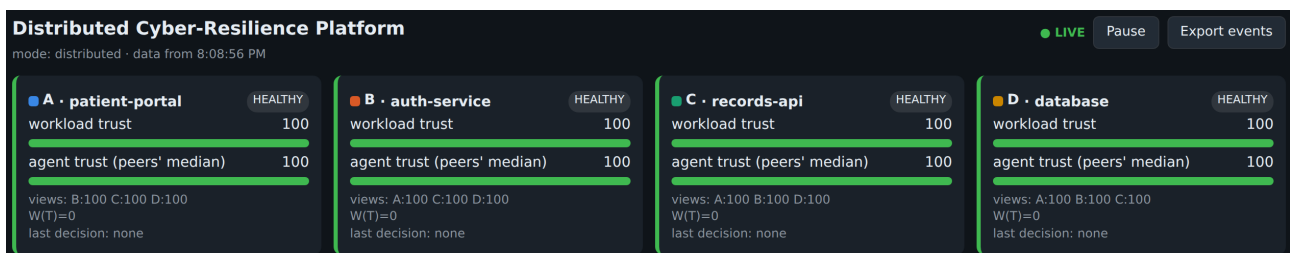


Distributed Cyber-Resilience Platform for Containerised Cloud Applications

A plain-English guide to explaining the project: what problem it solves, how it works, how to read the live dashboard, what every demo scenario shows, the results, the honest limits, and answers to the questions faculty are likely to ask.



The project in one sentence. Instead of one security “boss” that can be hacked or can crash, four independent security guards watch a cloud application, report to each other with sealed, signed messages, and only act when at least three of the four agree, and a guard that lies is automatically caught and ignored.

All screenshots in this guide were captured from the project's own dashboard. The numbers labelled **LIVE** were measured on the real 4-node Minikube cluster; numbers labelled **SIMULATOR** come from the built-in simulator, which runs the real agent code with faster timers.

Contents

1. The problem: one security boss is one weak spot	4
2. Our idea: four guards, no boss, three must agree	5
3. How it works, one idea at a time	6
3.1 Watching the apps	6
3.2 Sealed messages: digital signatures and mTLS	6
3.3 The weighted score $W(T)$: why one guard can never win alone	7
3.4 Trust: losing it and earning it back	7
3.5 The 3-of-4 vote	8
3.6 Isolation: the quarantine room	8
3.7 Recovery: rebuild from a clean master copy	8
3.8 Validation: prove the new copy is clean	9
3.9 The five reintegration stages	9
4. How to read the dashboard	10
4.1 The node cards	10
4.2 The live event timeline	10
4.3 A guard's own view (click a node card)	12
4.4 Pending votes: who has voted, who hasn't, and why	14
4.5 Decisions, and the evidence behind them	15
4.6 Rejected messages and incident metrics	17
5. The scenarios, step by step	18
5.1 Normal, healthy state	18
5.2 Genuine compromise of Node C (the full cycle)	18
5.3 A compromised guard falsely accuses a healthy node (the headline test)	21
5.4 A forgery attempt: pretending to be another guard	22
5.5 A guard crashes, then a real attack happens	23
5.6 The centralized baseline: one compromised controller	25
5.7 The centralized controller crashes, then a real attack happens	26
6. Results	28
7. A real bug we found and fixed	29
8. Honest limitations and future work	31
9. What we do NOT claim	32
10. Likely faculty questions, with short answers	33

11. Demo cheat sheet 35

12. Glossary 36

How to use this guide tomorrow.

- Sections 1-3 are your talk: the problem, the idea, and how it works, all with analogies.
- Section 4 teaches you (and the audience) to read the dashboard, so the demo makes sense.
- Section 5 walks through every scenario with screenshots, in case the live demo misbehaves.
- Section 11 is a one-page cheat sheet of the exact commands and what to say at each step. Print it.
- Sections 8-10 prepare you for questions: be open about limits, it earns trust.

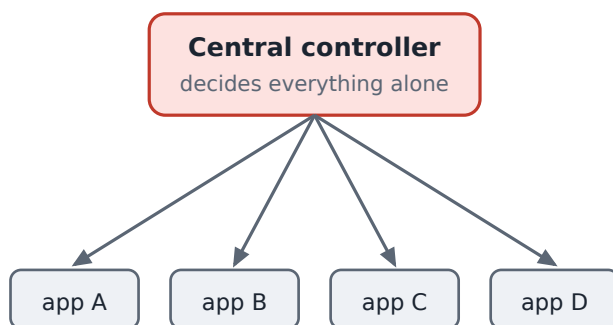
1. The problem: one security boss is one weak spot

Modern cloud applications are not one big program. They are built from many small, separate pieces, each running in its own **container** (a sealed, portable box that holds one piece of software and everything it needs to run). A hospital app, for example, might have one container for the patient website, one for logins, one for medical records and one for the database.

To protect such an app, most security tools use **one central controller**: a single program that watches every container, decides when something is under attack, and takes action, for example by cutting a container off from the network.

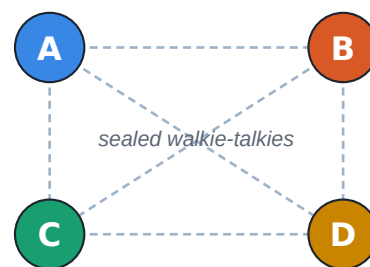
Everyday analogy. Picture a building with one head of security in a control room. Every alarm goes to them, and only they can lock a door. It works well, until that one person falls asleep, is tricked, or is bribed.

Today: one central controller



If it is hacked or crashes, nothing is protected, or healthy apps get locked by mistake.

Our design: four guards, no boss



Any lockdown needs 3 of the 4 guards to agree, so one bad or broken guard cannot decide alone.

This gives two dangers, and our experiments measured both:

- **Single point of failure** (one part whose failure stops the whole system): if the controller crashes, attacks go completely unnoticed. In our *controller-crash* test, a real attack ran for 8 seconds and the controller logged **nothing**.
- **Single point of compromise** (one part that, once hacked, gives the attacker control): if the controller is hacked, it can lock healthy parts of the app. On the **live cluster**, a compromised controller cut off the healthy login service within **about 4 seconds**, with nobody to object.

Takeaway: A security system that collapses when its own controller is attacked has not removed the weak spot; it has only moved it. We asked: what if the security system itself is designed to survive being attacked?

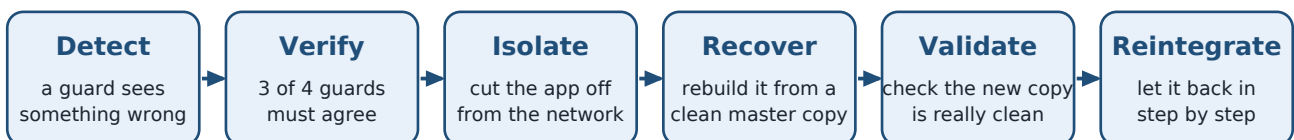
2. Our idea: four guards, no boss, three must agree

Everyday analogy. Replace the single head of security with **four guards**. Each guard patrols the whole building, not just one room. They carry **sealed walkie-talkies**: every message is stamped with the sender's personal seal, so nobody can fake another guard's voice. A room can only be locked when **at least three of the four** guards have seen the problem *with their own eyes* and agree. Guards also keep a private score of how believable each colleague is; a guard who keeps raising false alarms slowly loses credibility until the others stop listening to them.

In the project each guard is a small program we wrote called a **resilience agent**. There are four, named **A, B, C and D**. Each one runs on its own computer (a **node**) next to one part of a demo hospital application:

Node	Hospital part (the "workload")	What it does
A	patient-portal	the website patients and staff use
B	auth-service	checks usernames and passwords (logins)
C	records-api	serves medical records
D	database	stores the (fake, randomly generated) patient data

When something goes wrong, the guards take the app through a complete, automatic cycle:



Every step after "Detect" is decided or checked by at least 3 of the 4 guards.

- **No leader.** There is no boss guard. If any one guard crashes, the other three carry on.
- **Why exactly four?** Four is the smallest group that can survive one guard being dishonest. This is a well-known rule from computer science ($n = 3f + 1$): to tolerate f bad members you need $3f + 1$ members. With $f = 1$, that is 4 guards, and a decision needs $2f + 1 = 3$ votes.
- **Not only detection.** Many tools stop at raising an alarm. Ours also isolates, repairs, checks and carefully re-admits the app, and measures how long each step takes.

Takeaway: One bad or broken guard can neither force a wrong lockdown nor stop a right one, and a lying guard gets caught automatically.

3. How it works, one idea at a time

3.1 Watching the apps

Every second, each guard checks **all four** hospital parts (not just its own) and measures four simple signals. Each signal has a limit; going over the limit produces a **confidence** between 0.5 (just over) and 1.0 (clearly wrong). Confidence simply means “how sure am I that something is wrong”, from 0 to 1.

Signal	What is normal	Limit	What an attack looks like
Outbound traffic	a couple of connections, a few KiB/s	8 connections or 250 KiB/s	40 connections sending ~2 MiB/s: data being copied out
Processes	only the app itself (“python”)	anything not on the allowed list	a program called <i>xmrig</i> (a crypto-miner) running from a temporary folder
File fingerprints	every file matches its known-good fingerprint	any mismatch	the website's page file has been changed
Failed logins	none	10 in 10 seconds	dozens of wrong passwords coming from one part

A **file fingerprint** (technically a **hash**) is a short code calculated from a file's contents; change even one letter of the file and the fingerprint changes completely. We record the fingerprints of the clean files once, and compare against them.

The detection rules are deliberately simple thresholds. The project's contribution is not a clever detector; it is what happens *after* something is detected (Sections 3.2–3.9).

3.2 Sealed messages: digital signatures and mTLS

When a guard sees a problem, it writes a small report called **evidence**: *who* is reporting, *which part* looks wrong, *what kind* of signal, *how sure* it is, and *when*. It then protects the report in two ways:

- **A digital signature** (a mathematical seal that only the holder of a secret key can make, but anyone can check). Every guard has its own secret key; the others have its matching public key. We use a modern signature method called **Ed25519**. A forged or altered report fails the check.
- **mTLS** (mutual TLS: both sides of a connection show a certificate, like an ID badge, before they talk, and the conversation is encrypted). The badges are issued by our own small **certificate authority** (the “badge office”), so an outsider cannot even connect.

Everyday analogy. The signature is a **wax seal** on a letter. mTLS is **showing your ID badge at the door** before handing the letter over. A receiving guard only accepts a letter if the seal is genuine *and* the person handing it over is the same guard whose seal it bears.

Before accepting any report or vote, the receiving guard checks four things: the seal belongs to the badge-holder who delivered it; the seal is valid; the report names that

same guard as its author; and the report is fresh and not a copy of an old one (this blocks **replay attacks**, where an attacker re-sends an old genuine message). A message that fails costs the sender 20 trust points.

3.3 The weighted score $W(T)$: why one guard can never win alone

Each guard collects all reports about a target part T from the last 30 seconds and turns them into one number, the **weighted score $W(T)$** . It works in three small steps:

- **Weigh each report:** weight = confidence $\times$ (the sender's trust $\div$ 100). A report from a distrusted guard counts for less.
- **Per signal type, check support:** average the weights of everyone who reported that type. If only **one** guard reported it, the result is **halved** ("support" = 0.5); two or more guards give full support. One voice alone is never enough.
- **Combine:** W = the best signal-type score + a small bonus of 0.15 for each *extra* type that two or more guards confirmed. Different kinds of evidence agreeing is stronger than the same thing repeated.

A guard **votes to lock** (contain) T only if **both** are true: $W(T)$ is at least **0.6**, **and** it sees the problem **itself** (its own confidence is at least 0.5).

Situation	Working	$W(T)$	Result
Real attack on C, seen by all 4 guards on all 4 signals	best type 1.0 (network, 4 senders) + 3 extra confirmed types $\times$ 0.15	1.45	above 0.6, and every guard sees it itself $\rightarrow$ all vote
One lying guard, full trust, claims all 4 signals at 0.95	$0.95 \times 1.0 \times 0.5$ (only one sender) = 0.475; no bonus (nothing confirmed)	0.475	below 0.6 $\rightarrow$ cannot pass
The same liar after its trust fell to 2	$0.95 \times 0.02 \times 0.5$	0.012	far below

Takeaway: A single guard, however confident, can add at most 0.5 to the score, which is below the 0.6 bar. And even if it could, the other guards will not vote unless they see the problem themselves.

3.4 Trust: losing it and earning it back

Every guard keeps its own **trust score** (0 to 100) for each of the other three. Everyone starts at 100. The rules are simple straight-line changes (on the live cluster):

What happens	Effect on trust
A colleague's report is not confirmed by what I see myself (after an 8-second grace period, to allow for timing differences)	falls by 2.5 points every second
A colleague sends a forged or invalid message	falls by 20 points at once, per message
Nothing suspicious	recovers slowly: +0.5 points every second, up to 100

What happens	Effect on trust
Trust below 50	the colleague is marked SUSPECT on the dashboard
Trust below 40	the colleague's votes are ignored (the "vote-exclusion threshold")

That is why, on the live cluster, a lying guard is flagged in about 29–31 seconds: roughly 8 seconds of grace, then about 20 seconds to fall from 100 to 50, plus measurement timing. Trust falls fast and rises slowly on purpose, like trust between people.

Separately, every guard also keeps a **workload trust** for each hospital part: it drops to 0 when the part is isolated and climbs back at 2 points per second while the part behaves. It controls how fast a repaired part is let back in (Section 3.9).

3.5 The 3-of-4 vote

A vote is a signed message saying, for example, "I vote to CONTAIN C, incident number 1". When any guard holds **three matching signed votes** from three different guards it trusts, the decision is made. The bundle of three signed votes is called a **quorum certificate**, meaning a proof that a quorum (the minimum number needed) agreed, and it is written onto the action itself, so anyone can later check who authorised it.

Why 3 is safe: any two groups of 3 guards (out of 4) share at least 2 guards, and at most one guard is bad, so every decision includes at least one honest guard who really saw the problem.

Who actually does the work? All guards agree on a fixed order (A, then B, then C, then D). The first trusted guard in the order acts immediately; the others wait a few seconds and step in only if it has not happened. So if one guard is down, the next one does the job.

3.6 Isolation: the quarantine room

To isolate a part we use a Kubernetes **NetworkPolicy** (a firewall rule that says which parts are allowed to talk to which). **Kubernetes** is the standard system for running and managing containers, and **Calico** is the networking add-on that actually enforces those rules. In quarantine, only the guards can reach the part (so they can keep examining it); it cannot talk to anything.

Everyday analogy. A patient with a suspected infection is moved to an isolation room: doctors can still visit, but the patient cannot wander the hospital.

3.7 Recovery: rebuild from a clean master copy

We do not try to clean the infected container. We throw it away and start a brand-new one from the **known-good image** (a sealed master copy of the software, made before any attack). Anything the attacker changed or planted disappears with the old copy.

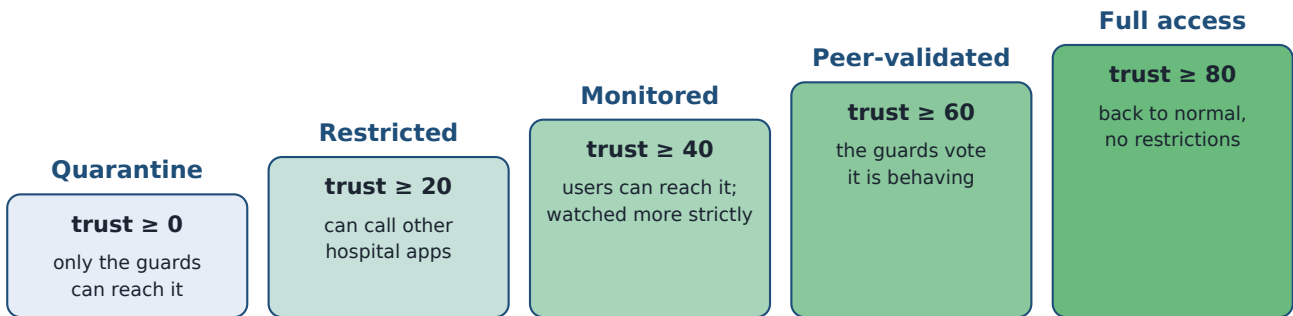
Everyday analogy. Instead of scrubbing a laptop that has a virus, you wipe it and reinstall from the original installation disk.

3.8 Validation: prove the new copy is clean

Each guard independently checks the new copy: it answers its health check; it really is a **new** copy, not the old one; every file fingerprint matches the known-good list; and none of the four signals is over its limit. Only when three guards vote “VALIDATE” does the part move on.

3.9 The five reintegration stages

A repaired part is **not** let straight back in. It climbs five steps, each with more freedom:



Each step up needs: at least 10 seconds on the current step + enough trust + no new anomaly + a 3-of-4 vote.

Everyday analogy. Like leaving hospital after serious surgery: intensive care, then a ward, then an observation room, then the doctors' sign-off, and only then home. Each move needs time, improvement and a doctor's agreement.

If any new problem appears on the way up, the guards can vote to isolate it again immediately.

4. How to read the dashboard

The dashboard is a web page that **only shows** what the guards are doing; it never makes decisions. It refreshes every second. From top to bottom it has: the node cards, the live event timeline, the decisions table, pending votes, rejected messages and incident metrics. Clicking a node card opens that guard's own view.

4.1 The node cards

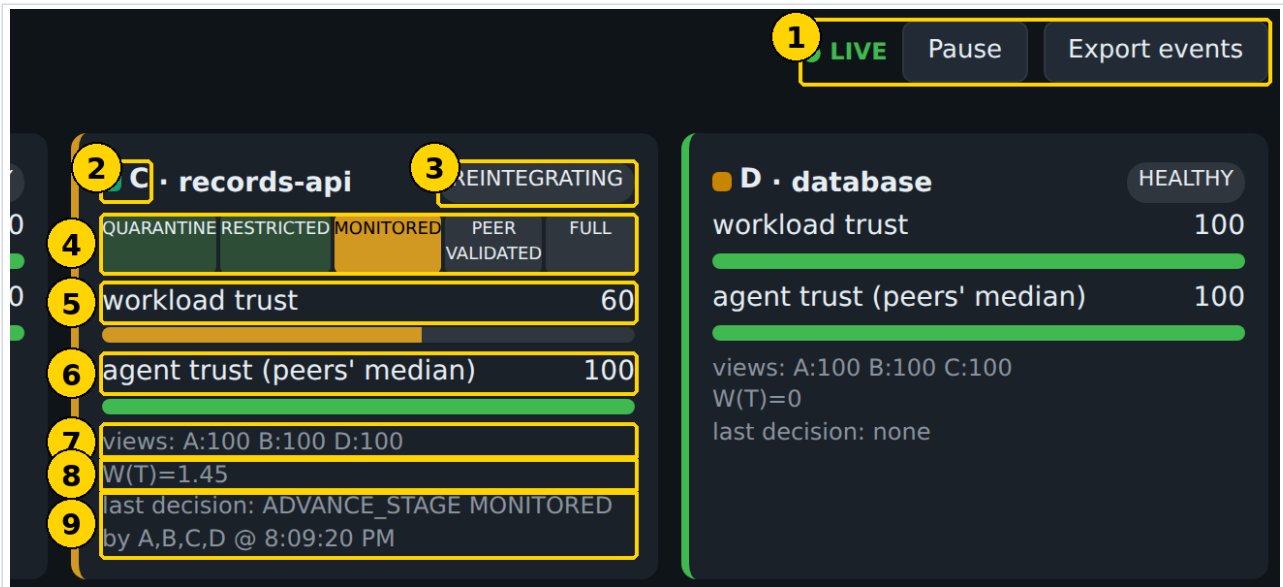


Figure 1. Node C's card during reintegration (with the toolbar), with numbered callouts.

What you're looking at

- **1 · Toolbar.** “● LIVE” means the page is updating. **Pause** freezes the whole screen while you explain something (data is still collected and appears when you press Resume). **Export events** downloads every guard's event log as a file for the report.
- **2 · Which node.** The coloured square and letter (C) plus the hospital part it hosts (records-api). The *letter* is what identifies a node everywhere; the colour is only a helper.
- **3 · State of the hospital part.** HEALTHY, ISOLATED, RECOVERING, VALIDATING or REINTEGRATING. The card's left border matches: green healthy, red isolated or recovering, amber validating or reintegrating.
- **4 · Reintegration steps.** Only shown while a part is returning. Dark green = steps completed (Quarantine, Restricted); amber = the current step (**Monitored**); grey = still to come.
- **5 · Workload trust = 60.** How healthy the guards believe records-api is. It was reset to 0 when isolated and is climbing back (2 points per second on the live cluster).
- **6 · Agent trust = 100.** How much the *other three* guards believe guard C (the middle value of their three opinions). Below 50 a red **SUSPECT** tag appears here.
- **7 · Views.** Each other guard's own opinion of guard C: A:100 B:100 D:100.
- **8 · W(T)=1.45.** The weighted evidence score about this part (Section 3.3); here it still shows the last score computed during the attack.
- **9 · Last decision.** The most recent agreed action on this part and **who signed it**: “move to MONITORED, signed by A, B, C, D”.

4.2 The live event timeline

Every guard narrates what it does, one plain sentence per event, newest first. Because each guard logs *its own* view, the same moment usually appears three or four times, once per guard. That repetition is the point: it shows independent guards reaching the same conclusion.

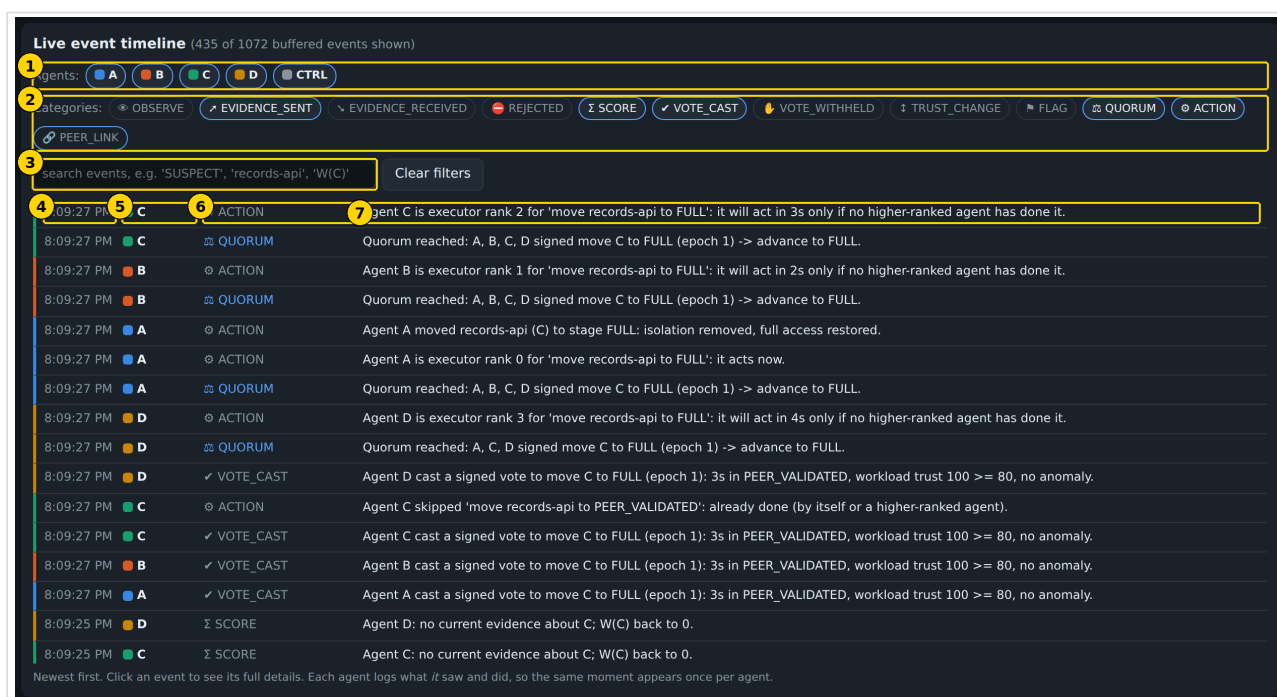


Figure 2. The event timeline, filtered to decision-related events at the end of Node C's recovery.

What you're looking at

- 1 • Agent filter.** Click a letter to hide or show that guard's events (CTRL is the centralized controller, used only in baseline mode).
- 2 • Category filter.** Highlighted chips are shown. Here only EVIDENCE_SENT, SCORE, VOTE_CAST, QUORUM, ACTION and PEER_LINK are selected, which hides the many routine observations.
- 3 • Search.** Type any word, e.g. *SUSPECT*, *records-api* or *W(C)*.
- 4 • Time** the event happened.
- 5 • Which guard** wrote it.
- 6 • Category** with its icon (see the table below).
- 7 • The sentence.** Read the rows from the bottom up: all four guards cast signed votes to move C to FULL; each guard sees “Quorum reached”; guard A (first in the order) says it “acts now” and “moved records-api (C) to stage FULL: isolation removed”; B, C and D record that they would only act if A had not, and then “skipped ... already done”.

Icon	Category	Read it as
(eye)	OBSERVE	“Here is what I measured on this part”: the 4 signals against their limits
↗	EVIDENCE_SENT	“I signed a report of what I saw and sent it to the others”
↘	EVIDENCE_RECEIVED	“I received a colleague's report; its seal and badge check out”
(stop)	REJECTED	“I threw a message away”: forged seal or wrong badge
Σ	SCORE	“Adding up all the reports, weighted by who sent them, W(T) = ...”

Icon	Category	Read it as
✓	VOTE_CAST	"I vote yes, because ..."
(hand)	VOTE_WITHHELD	"I am <i>not</i> voting, because ..." (usually: I don't see it myself)
↕	TRUST_CHANGE	"My trust in that guard went from X to Y, because ..."
🚩	FLAG	"I now consider that guard SUSPECT" or "I stop counting its votes"
⚖️	QUORUM	"3 of 4 guards signed it: the decision is made"
⚙️	ACTION	"I applied the isolation / started the rebuild / changed the step", or skipped it because another guard already did
(link)	PEER_LINK	"I lost (or regained) my connection to another guard"

4.3 A guard's own view (click a node card)

Clicking a card opens a side panel showing the world *through that guard's eyes*. It answers four questions: what does it measure, whom does it trust, how does it score the evidence, and why has it voted or not.

What it measures on each workload (value vs threshold)					
workload	outbound traffic	processes	file hashes	failed logins	overall
■ A patient-portal	2 conns / limit 8 1 KiB/s / limit 250 conf 0	1 running, all allowed conf 0	3 checked, all match conf 0	0 in 10s / limit 10 conf 0	0.00
■ B auth-service	2 conns / limit 8 1 KiB/s / limit 250 conf 0	1 running, all allowed conf 0	3 checked, all match conf 0	0 in 10s / limit 10 conf 0	0.00
■ C records-api	2 conns / limit 8 1 KiB/s / limit 250 conf 0	1 running, all allowed conf 0	3 checked, all match conf 0	0 in 10s / limit 10 conf 0	0.00
■ D database	2 conns / limit 8 1 KiB/s / limit 250 conf 0	1 running, all allowed conf 0	3 checked, all match conf 0	0 in 10s / limit 10 conf 0	0.00

Figure 3. Guard B's measurements of all four hospital parts, in normal operation.

What you're looking at

- One row per hospital part (A patient-portal, B auth-service, C records-api, D database); one column per signal, each shown as **measured value / limit**.
- Outbound traffic: "2 conns / limit 8" and "1 KiB/s / limit 250": far below the limits.
- Processes: "1 running, all allowed". File hashes: "3 checked, all match". Failed logins: "0 in 10s / limit 10".
- "conf 0" under each value and **overall 0.00**: guard B sees nothing wrong anywhere. Compare with Figure 13, taken during the attack, where the whole C row turns red.

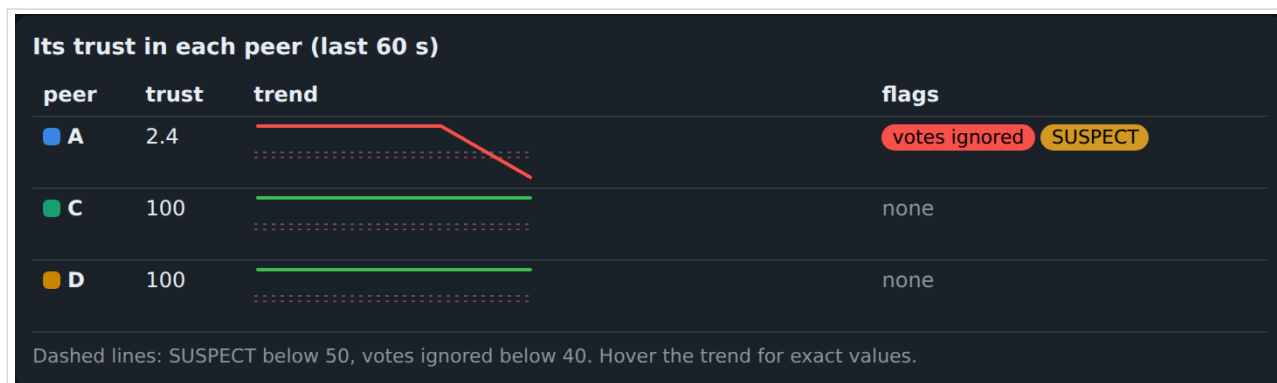


Figure 4. Guard B's trust in each colleague over the last 60 seconds, during the false-accusation scenario.

What you're looking at

- Row A: trust **2.4**. The red line was flat at 100, then drops in a straight line once A's false claims start being contradicted.
- The two dashed lines mark 50 (SUSPECT) and 40 (votes ignored). The line crossed both.
- Flags column: **votes ignored** and **SUSPECT** for A; **none** for C and D, whose lines stay flat and green at 100.
- On the dashboard you can hover over a line to read the exact value at any moment.

Evidence it holds and its weighted score $W(T)$ per workload

■ **A** $W(A) = 0$ (votes to contain at ≥ 0.6 , only if it also sees the anomaly itself)

■ **B** $W(B) = 0.012$ (votes to contain at ≥ 0.6 , only if it also sees the anomaly itself)

type	sender	confidence	× sender trust	= weight
NETWORK	■ A	0.95	0.02	0.023
PROCESS	■ A	0.95	0.02	0.023
FILE_INTEGRITY	■ A	0.95	0.02	0.023
AUTH	■ A	0.95	0.02	0.023
type	mean weight	senders	× support	= type score
NETWORK	0.023	1	0.5	0.012
PROCESS	0.023	1	0.5	0.012
FILE_INTEGRITY	0.023	1	0.5	0.012
AUTH	0.023	1	0.5	0.012

$W = \text{best type score} + \text{diversity bonus } 0 \text{ (corroborated types: none)} = \mathbf{0.012}$

■ **C** $W(C) = 0$ (votes to contain at ≥ 0.6 , only if it also sees the anomaly itself)

■ **D** $W(D) = 0$ (votes to contain at ≥ 0.6 , only if it also sees the anomaly itself)

evidence pool (last 20)

8:09:57 PM	■ A→B	AUTH	0.95	[fabricated] auth anomaly
8:09:57 PM	■ A→B	FILE_INTEGRITY	0.95	[fabricated] file_integrity anomaly
8:09:57 PM	■ A→B	PROCESS	0.95	[fabricated] process anomaly
8:09:57 PM	■ A→B	NETWORK	0.95	[fabricated] network anomaly

Figure 5. Guard B's evidence and weighted score $W(B)$ during the false accusation (cropped).

What you're looking at

- **W(B) = 0.012**: the score about Node B, far below the 0.6 needed.
- The first table lists each report: all four come from **A only**, each with confidence 0.95 but **sender trust 0.02** (A's trust of 2 out of 100), giving a weight of just 0.023.
- The second table: each signal type has **1 sender**, so it gets only **half support (0.5)**, giving a type score of 0.012.
- “corroborated types: none” means no other guard confirmed anything, so there is no bonus.
- The evidence pool at the bottom shows the reports themselves, each marked **[fabricated]** by our simulation so the audience can see they are fake.

Its votes and why

proposal	its vote	reason
CONTAIN A (epoch 1)	not voting	own observation of A normal (confidence 0.00 < 0.5)
CONTAIN B (epoch 1)	not voting	own observation of B normal (confidence 0.00 < 0.5)
CONTAIN C (epoch 2)	not voting	C is already being handled (phase RECOVERING)
CONTAIN D (epoch 1)	not voting	own observation of D normal (confidence 0.00 < 0.5)

Figure 6. Guard B's votes and the reason for each, a moment after C was contained.

What you're looking at

- One row per possible proposal, with B's current stance and its exact reason.
- CONTAIN A, B and D: **not voting**, “own observation of ... normal (confidence 0.00 < 0.5)”: B checked each of them itself and saw nothing wrong.
- CONTAIN C (**epoch 2**): **not voting**, “C is already being handled (phase RECOVERING)”. B already voted yes for incident 1 a second earlier, and that vote succeeded; a second incident makes no sense while C is being rebuilt.
- Seconds before this, the same row read “votes yes: sees the anomaly itself (confidence 1.00) and W(C)=1.45 ≥ 0.6”, and the timeline records that as a VOTE_CAST event.

4.4 Pending votes: who has voted, who hasn't, and why

Pending votes (need 3 of 4 eligible signed votes; each agent says why it has not voted)

CONTAIN B (epoch 1) — 1/3 signed (A)

A	✓ voted	
B	not voted	own observation of B normal (confidence 0.00 < 0.5)
C	not voted	own observation of B normal (confidence 0.00 < 0.5)
D	not voted	own observation of B normal (confidence 0.00 < 0.5)
	not counted	B does not count: A's vote: trust below vote-exclusion threshold 40 (trust 32.5)
	not counted	C does not count: A's vote: trust below vote-exclusion threshold 40 (trust 32.5)
	not counted	D does not count: A's vote: trust below vote-exclusion threshold 40 (trust 32.5)

Figure 7. The pending-votes panel while compromised guard A tries to get healthy Node B locked.

What you're looking at

- Title: **CONTAIN B (epoch 1) — 1/3 signed (A)**. “Epoch 1” means incident number 1 for Node B. Only A has signed; 3 are needed.
- Rows B, C, D: **not voted**, each with its own reason: “own observation of B normal (confidence $0.00 < 0.5$)”. Each guard looked for itself and saw nothing wrong.
- The three amber **not counted** rows: B, C and D each say they do not count A's vote at all, because A's trust (32.4 to 32.5) is below the vote-exclusion threshold of 40.
- So this proposal can never pass: it is stuck at 1 of 3, and even that one vote is ignored.

4.5 Decisions, and the evidence behind them

Recent quorum decisions (click a row to see the evidence and votes that justified it)						
time	action	target	epoch	stage	signed by	
8:09:27 PM	ADVANCE_STAGE	C	1	FULL	   	► why?
8:09:24 PM	ADVANCE_STAGE	C	1	PEER_VALIDATED	   	► why?
8:09:20 PM	ADVANCE_STAGE	C	1	MONITORED	   	► why?
8:09:17 PM	ADVANCE_STAGE	C	1	RESTRICTED	  	► why?
8:09:13 PM	VALIDATE	C	1		  	► why?
8:09:05 PM	CONTAIN	C	1		   	► why?

Figure 8. Every decision taken during Node C's full recovery.

What you're looking at

- Read from the bottom up: **CONTAIN** (isolate and rebuild), **VALIDATE** (the new copy is clean), then four **ADVANCE_STAGE** steps: RESTRICTED, MONITORED, PEER_VALIDATED and FULL.
- “Signed by” lists the guards whose signed votes made up the quorum certificate. Some rows show 3 signers (e.g. A, C, D for VALIDATE), others 4: three is enough, and the fourth vote sometimes arrives before the count is taken.
- Every row has ► **why?**. Clicking it expands the evidence (next figure).

Recent quorum decisions (click a row to see the evidence and votes that justified it)						
time	action	target	epoch	stage	signed by	
8:09:05 PM	CONTAIN	C	1		A B C D	▼ hide
isolate + recover · seen by A B C D						
signed vote		voter's W(T)		signed at	evidence cited	
A		1.3		8:09:05 PM	16	
B		1.3		8:09:05 PM	17	
C		1.3		8:09:05 PM	17	
D		1.3		8:09:05 PM	17	
Evidence behind it						
from	type	confidence		what was seen		
A	NETWORK	1		40 outbound conns, tx 376 KiB/s		
A	PROCESS	0.95		unexpected processes: xmrig		
A	FILE_INTEGRITY	0.95		1 modified file(s): static/index.html		
B	PROCESS	0.95		unexpected processes: xmrig		
B	NETWORK	1		40 outbound conns, tx 379 KiB/s		
C	NETWORK	1		40 outbound conns, tx 381 KiB/s		
C	PROCESS	0.95		unexpected processes: xmrig		
D	PROCESS	0.95		unexpected processes: xmrig		
D	NETWORK	1		40 outbound conns, tx 382 KiB/s		
B	FILE_INTEGRITY	0.95		1 modified file(s): static/index.html		
C	FILE_INTEGRITY	0.95		1 modified file(s): static/index.html		
D	FILE_INTEGRITY	0.95		1 modified file(s): static/index.html		
A	NETWORK	1		40 outbound conns, tx 1954 KiB/s		
A	PROCESS	0.95		unexpected processes: xmrig		
A	FILE_INTEGRITY	0.95		1 modified file(s): static/index.html		
A	AUTH	0.512		11 failed auths from 10.244.0.12 in 10s		
C	NETWORK	1		40 outbound conns, tx 1954 KiB/s		
B	NETWORK	1		40 outbound conns, tx 1954 KiB/s		
C	PROCESS	0.95		unexpected processes: xmrig		
B	PROCESS	0.95		unexpected processes: xmrig		
D	NETWORK	1		40 outbound conns, tx 1954 KiB/s		
C	FILE_INTEGRITY	0.95		1 modified file(s): static/index.html		
D	PROCESS	0.95		unexpected processes: xmrig		
B	FILE_INTEGRITY	0.95		1 modified file(s): static/index.html		
C	AUTH	0.512		11 failed auths from 10.244.0.12 in 10s		
B	AUTH	0.512		11 failed auths from 10.244.0.12 in 10s		
D	FILE_INTEGRITY	0.95		1 modified file(s): static/index.html		
D	AUTH	0.525		12 failed auths from 10.244.0.12 in 10s		
type		sender	confidence	× sender trust	= weight	
NETWORK		A	1	1.00	1	
NETWORK		B	1	1.00	1	
NETWORK		C	1	1.00	1	
NETWORK		D	1	1.00	1	
PROCESS		A	0.95	1.00	0.95	
PROCESS		B	0.95	1.00	0.95	
PROCESS		C	0.95	1.00	0.95	
PROCESS		D	0.95	1.00	0.95	
FILE_INTEGRITY		A	0.95	1.00	0.95	
FILE_INTEGRITY		B	0.95	1.00	0.95	
FILE_INTEGRITY		C	0.95	1.00	0.95	
FILE_INTEGRITY		D	0.95	1.00	0.95	
AUTH		A	0.637	1.00	0.637	
AUTH		C	0.512	1.00	0.512	
AUTH		B	0.512	1.00	0.512	
AUTH		D	0.525	1.00	0.525	
type		mean weight	senders	× support	= type score	
NETWORK		1	4	1	1	
PROCESS		0.95	4	1	0.95	
FILE_INTEGRITY		0.95	4	1	0.95	
AUTH		0.546	4	1	0.546	
W = best type score + diversity bonus 0.45 (corroborated types: AUTH, FILE_INTEGRITY, NETWORK, PROCESS) = 1.45						

Figure 9. The CONTAIN decision for Node C, expanded to show its justification.

What you're looking at

- Top: the four **signed votes**. Each voter's own score $W(T)$ was 1.3 when it voted; each cited 16–17 pieces of evidence.
- **Evidence behind it**: every report, from every guard: NETWORK “40 outbound conns, tx 1954 KiB/s”, PROCESS “unexpected processes: xmrig”, FILE_INTEGRITY “1 modified file(s): static/index.html”, and AUTH “11 failed auths from 10.244.0.12 in 10s”.
- Bottom two tables: the score calculation. Every type has **4 senders** and full support (1); $W = \text{best type score } 1.0 + \text{diversity bonus } 0.45 \text{ (3 extra confirmed types} \times 0.15) = \mathbf{1.45}$.
- This is the audit trail: for any decision you can show exactly who agreed and what they saw.

4.6 Rejected messages and incident metrics

Rejected messages (signature / mTLS identity failures)

time	from (mTLS)	claimed	reason	seen by
8:12:19 PM	A	B	signer 'B' != mTLS identity 'A'	B
8:12:19 PM	A	B	signer 'B' != mTLS identity 'A'	C
8:12:19 PM	A	B	signer 'B' != mTLS identity 'A'	D
8:12:18 PM	A	B	signer 'B' != mTLS identity 'A'	C
8:12:18 PM	A	B	signer 'B' != mTLS identity 'A'	D
8:12:18 PM	A	B	signer 'B' != mTLS identity 'A'	B
8:12:18 PM	A	B	signer 'B' != mTLS identity 'A'	D
8:12:18 PM	A	B	signer 'B' != mTLS identity 'A'	B
8:12:18 PM	A	B	signer 'B' != mTLS identity 'A'	C

Figure 10. The rejected-messages table during the forgery scenario.

What you're looking at

- Each row is a message a guard refused. **From (mTLS)**: the badge of whoever delivered it (A). **Claimed**: whose seal the message pretended to carry (B).
- **Reason**: “signer 'B' != mTLS identity 'A'”, meaning the letter claims to be from B but A handed it over. **Seen by**: the guard that caught it (B, C and D all did).

Incident metrics (seconds since injection; min across agents)

scenario	target	mode	TTD	TTI	TTR	validated	full	trust-recovery	flag attacker	false isolation	availability
app-compromise	C	distributed	0.1	1.1	8.1	9.1	22.6	13.5	-	no	-
false-accusation	B	distributed	-	-	-	-	-	-	13.9	no	-

Figure 11. The incident metrics table (simulator, this photo session).

What you're looking at

- One row per test run (“incident”). Times are in seconds from the moment the problem started.
- **TTD** = time to detect; **TTI** = time to isolate; **TTR** = time to recover (clean copy running); **validated**; **full** = back to full access; **trust-recovery** = time spent climbing the five steps; **flag attacker** = time until a lying guard was marked SUSPECT; **false isolation** = whether a healthy part was wrongly locked.
- Here: the attack on C was detected in 0.1 s and isolated in 1.1 s; the false accusation flagged the liar after 13.9 s with **no** false isolation.
- For these photos the rebuild step was deliberately slowed so each stage could be captured; Section 6 gives the standard figures.

5. The scenarios, step by step

The genuine-compromise, agent-crash and controller-crash scenarios are shown with the **simulator**: the real guard programs, real signatures, real encrypted connections and real voting, running on one machine with a pretend cluster and faster timers (half-second checks, 3-second minimum per step). The false-accusation, forgery and baseline scenarios also run on the **live cluster**; their live numbers are in Section 6.

5.1 Normal, healthy state

What we do: nothing: start everything and wait.

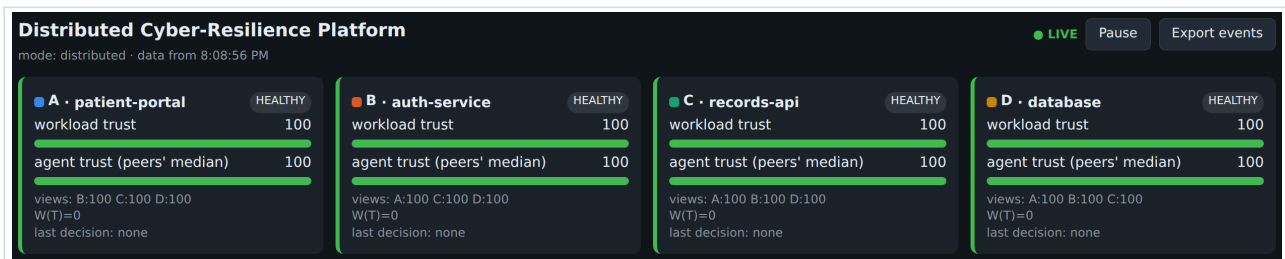


Figure 12. All four hospital parts healthy; all guards trust each other.

What you're looking at

- All four cards say **HEALTHY** with green borders.
- Workload trust 100 and agent trust 100 everywhere; every view reads 100.
- $W(T)=0$ on every card: nobody holds any evidence of a problem. "Last decision: none".
- At the top right, "● LIVE" shows the page is updating every second.

Takeaway: When nothing is wrong, the guards stay quiet: no evidence, no votes, no actions.

5.2 Genuine compromise of Node C (the full cycle)

What we do: make records-api (C) behave like a hacked service: it starts copying data out over 40 connections, runs a hidden crypto-miner called *xmrig*, has a web page file altered, and hammers the login service with wrong passwords. (On the live cluster this step is shown with the simulator; see Section 8.)

What happens, step by step:

- **Detect** (within half a second): all four guards measure C's signals over their limits.
- **Evidence:** each guard signs and sends reports on all four signal types.
- **Score:** every guard computes $W(C) = 1.45$, far above 0.6, and sees the problem itself.
- **Vote:** all four vote CONTAIN C; three signatures are enough and the decision is made.
- **Isolate and recover:** guard A (first in the order) applies the quarantine rule and starts a clean copy from the master image; B, C and D check and skip it because it is already done.
- **Validate:** each guard checks the new copy; three vote VALIDATE.

- **Reintegrate:** four more 3-of-4 votes walk C through Restricted, Monitored, Peer-validated and Full.

What it measures on each workload (value vs threshold)					
workload	outbound traffic	processes	file hashes	failed logins	overall
■ A patient-portal	2 conns / limit 8 1 KiB/s / limit 250 conf 0	1 running, all allowed conf 0	3 checked, all match conf 0	0 in 10s / limit 10 conf 0	0.00
■ B auth-service	2 conns / limit 8 1 KiB/s / limit 250 conf 0	1 running, all allowed conf 0	3 checked, all match conf 0	0 in 10s / limit 10 conf 0	0.00
■ C records-api	40 conns / limit 8 1954 KiB/s / limit 250 conf 1	2 running unexpected: xmrig conf 0.95	3 checked 1 modified, 0 missing, 0 new conf 0.95	31 in 10s / limit 10 conf 0.762	1.00
■ D database	2 conns / limit 8 1 KiB/s / limit 250 conf 0	1 running, all allowed conf 0	3 checked, all match conf 0	0 in 10s / limit 10 conf 0	0.00

Figure 13. Guard B's measurements seconds after the attack starts.

What you're looking at

- Row C (records-api) is **red in every column**: “40 conns / limit 8” and “1954 KiB/s / limit 250”; “2 running, unexpected: xmrig”; “1 modified” file; “31 in 10s / limit 10” failed logins.
- Confidences: 1 (network), 0.95 (process), 0.95 (files), 0.762 (logins). Overall **1.00**.
- Rows A, B and D stay at 0.00: the problem is exactly where the attack is.

Live event timeline (921 of 921 buffered events shown)		
Agents: ■ A ■ B ■ C ■ D ■ CTRL		
Categories: ⊕ OBSERVE ➤ EVIDENCE_SENT ▼ EVIDENCE_RECEIVED ⊖ REJECTED Σ SCORE ✓ VOTE_CAST ⚡ VOTE_WITHHELD ⚡ TRUST_CHANGE 🚩 FLAG ⚖ QUORUM ⚙ ACTION		
PEER_LINK		
search events, e.g. 'SUSPECT', 'records-api', 'W(C)' Clear filters		
8:09:10 PM	■ D	⊙ ACTION Agent D skipped 'redploy records-api from known-good image': already done (by itself or a higher-ranked agent).
8:09:10 PM	■ D	⊙ ACTION Agent D skipped 'isolate records-api': already done (by itself or a higher-ranked agent).
8:09:08 PM	■ C	⊙ ACTION Agent C skipped 'redploy records-api from known-good image': already done (by itself or a higher-ranked agent).
8:09:08 PM	■ C	⊙ ACTION Agent C skipped 'isolate records-api': already done (by itself or a higher-ranked agent).
8:09:07 PM	■ D	⊙ OBSERVE Agent D sees records-api (C): 40 outbound connections (limit 8) and sending 1954 KiB/s (limit 250 KiB/s); unexpected process xmrig; 1 file differing from the known-good hashes; 52 failed logins in 10s (limit 10) -> confidence 1.00.
8:09:07 PM	■ C	⊙ OBSERVE Agent C sees records-api (C): 40 outbound connections (limit 8) and sending 1954 KiB/s (limit 250 KiB/s); unexpected process xmrig; 1 file differing from the known-good hashes; 51 failed logins in 10s (limit 10) -> confidence 1.00.
8:09:07 PM	■ B	⊙ ACTION Agent B skipped 'redploy records-api from known-good image': already done (by itself or a higher-ranked agent).
8:09:07 PM	■ B	⊙ ACTION Agent B skipped 'isolate records-api': already done (by itself or a higher-ranked agent).
8:09:07 PM	■ B	⊙ OBSERVE Agent B sees records-api (C): 40 outbound connections (limit 8) and sending 1954 KiB/s (limit 250 KiB/s); unexpected process xmrig; 1 file differing from the known-good hashes; 51 failed logins in 10s (limit 10) -> confidence 1.00.
8:09:07 PM	■ A	⊙ OBSERVE Agent A sees records-api (C): 40 outbound connections (limit 8) and sending 1954 KiB/s (limit 250 KiB/s); unexpected process xmrig; 1 file differing from the known-good hashes; 51 failed logins in 10s (limit 10) -> confidence 1.00.
8:09:06 PM	■ D	⊙ OBSERVE Agent D sees records-api (C): 40 outbound connections (limit 8) and sending 1954 KiB/s (limit 250 KiB/s); unexpected process xmrig; 1 file differing from the known-good hashes; 42 failed logins in 10s (limit 10) -> confidence 1.00.
8:09:06 PM	■ C	⊙ OBSERVE Agent C sees records-api (C): 40 outbound connections (limit 8) and sending 1954 KiB/s (limit 250 KiB/s); unexpected process xmrig; 1 file differing from the known-good hashes; 41 failed logins in 10s (limit 10) -> confidence 1.00.
8:09:06 PM	■ B	⊙ OBSERVE Agent B sees records-api (C): 40 outbound connections (limit 8) and sending 1954 KiB/s (limit 250 KiB/s); unexpected process xmrig; 1 file differing from the known-good hashes; 41 failed logins in 10s (limit 10) -> confidence 1.00.

Figure 14. The timeline right after containment: four guards, one conclusion.

What you're looking at

- Many **OBSERVE** rows, one per guard (A, B, C, D), all saying the same thing: “sees records-api (C): 40 outbound connections (limit 8) ... unexpected process xmgrig; 1 file differing ... 51 failed logins in 10s (limit 10) -> confidence 1.00”.
- Four independent observers agreeing is exactly what the 3-of-4 rule relies on.
- At the top, **ACTION** rows from B, C and D: “skipped 'isolate records-api': already done” and “skipped 'redeploy ...': already done”. Guard A did the work, and the others confirmed and stood back.

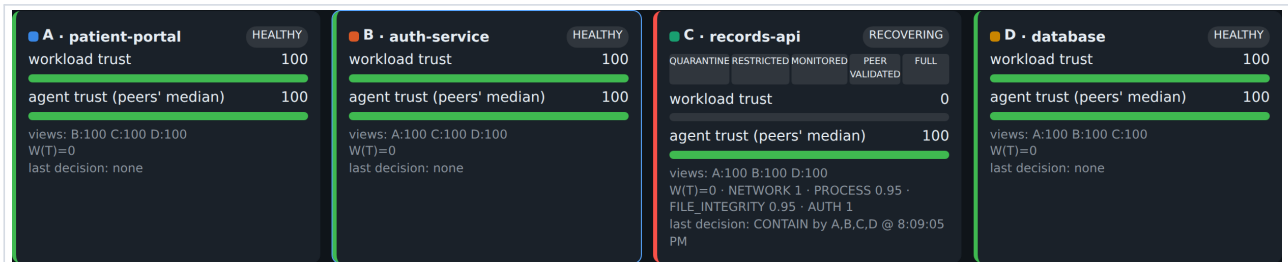


Figure 15. Node C isolated and being rebuilt.

What you're looking at

- C's card: **RECOVERING**, red border; the five-step bar is visible (nothing done yet).
- Workload trust **0**: reset on isolation.
- The detections line lists what triggered it: NETWORK 1 · PROCESS 0.95 · FILE_INTEGRITY 0.95 · AUTH 1.
- “Last decision: CONTAIN by A,B,C,D”: all four guards signed.
- A, B and D stay **HEALTHY**: only the attacked part was touched.

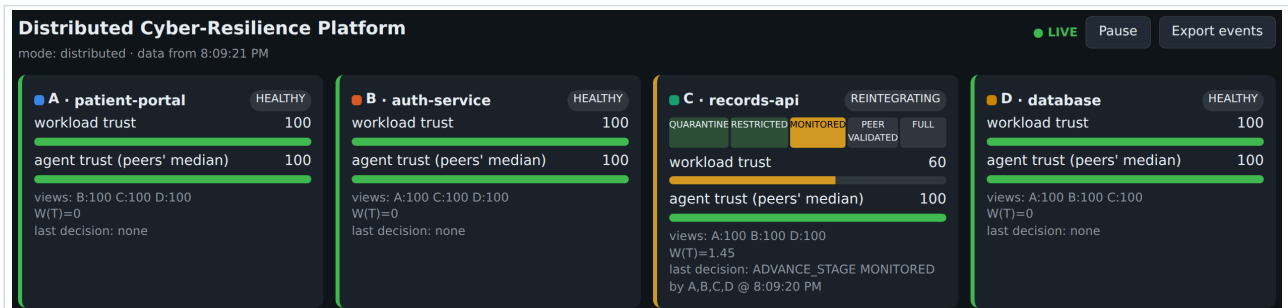


Figure 16. Half-way back: Node C at the MONITORED step.

What you're looking at

- C's card: **REINTEGRATING**, amber border. Quarantine and Restricted are dark green (done), **Monitored** is amber (current step), Peer-validated and Full are still grey.
- Workload trust **60** and rising: it has just reached the 60 needed for the next step; full access needs 80.
- “Last decision: ADVANCE_STAGE MONITORED by A,B,C,D”.

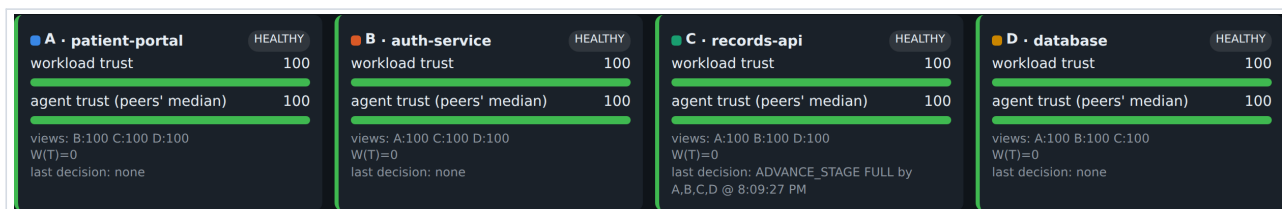


Figure 17. The end: Node C healthy again with full access.

What you're looking at

- All four cards are **HEALTHY** again; C's workload trust is back to 100.
- C's "last decision" is "ADVANCE_STAGE FULL by A,B,C,D": the final step, also voted.
- The complete list of decisions is shown in Figure 8.

Takeaway: A real attack is caught by all the guards independently, contained in about a second, rebuilt from a clean copy, checked, and carefully re-admitted, with every step agreed by at least three guards and recorded.

5.3 A compromised guard falsely accuses a healthy node (the headline test)

What we do: switch guard A into "compromised insider" mode. It keeps its real key and badge (so its messages are genuinely signed), but it invents high-confidence reports of all four signal types against the **healthy** login service B, and keeps voting to lock B.

What happens:

- B, C and D receive A's reports; the seals are valid, so they are accepted as reports, *not* as truth.
- Each honest guard checks B for itself and sees nothing wrong, so it **refuses to vote** ("own observation of B normal").
- After the 8-second grace period, A's claims count as contradicted and A's trust falls in a straight line.
- Below 50 A is marked **SUSPECT**; below 40 its votes are **ignored**.
- The proposal to lock B stays at 1 of 3 forever. B is never isolated, and the app keeps working.

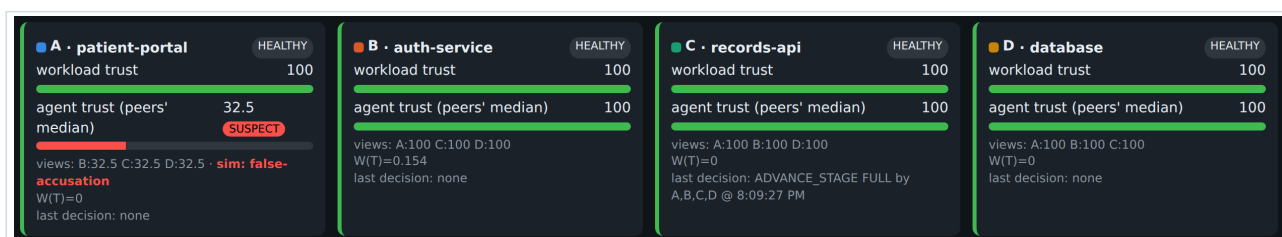


Figure 18. Guard A exposed: agent trust 32.5 and a SUSPECT tag.

What you're looking at

- A's card: agent trust **32.5**, red bar, red **SUSPECT** tag. Views: B:32.5 C:32.5 D:32.5: all three honest guards independently reached the same low opinion.
- “sim: false-accusation” in red tells the audience that this guard is the one we deliberately compromised.
- B's card: still **HEALTHY**, workload trust 100. $W(T)=0.154$: A's reports reach it, but at a tiny weight, well below 0.6.
- No card shows a new decision: nothing was locked.

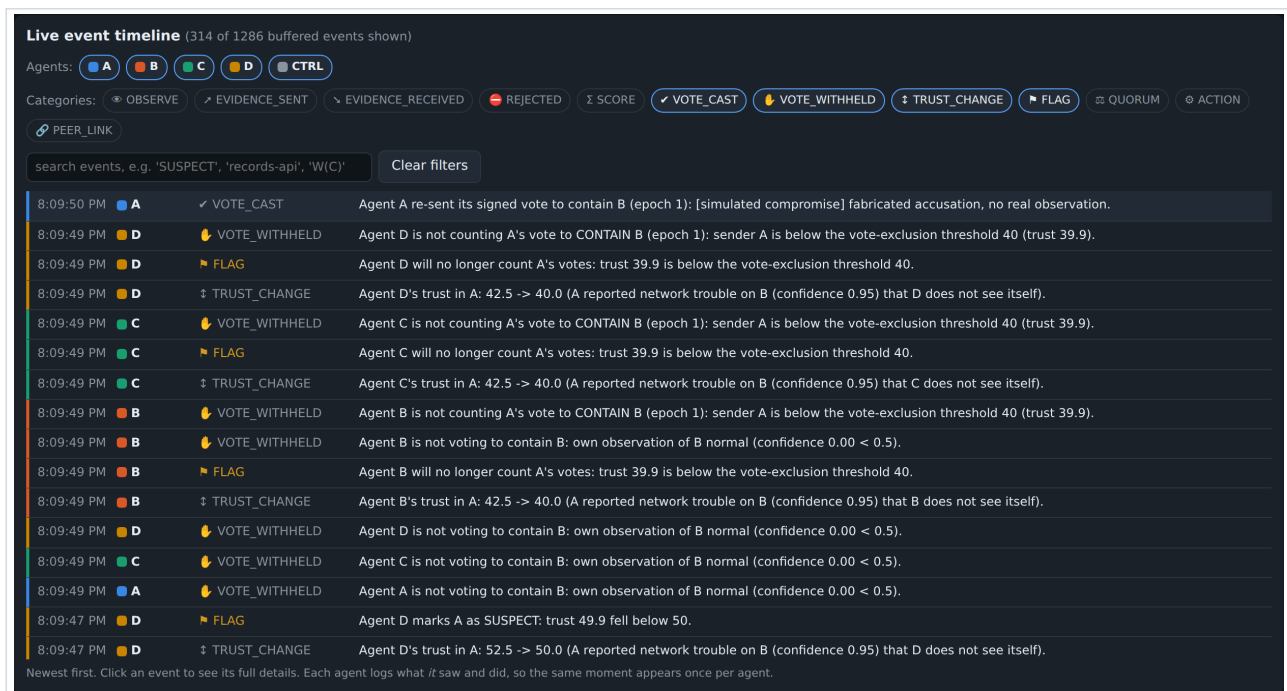


Figure 19. The honest guards' reasoning, filtered to trust, flags and votes.

What you're looking at

- Bottom rows: “Agent D's trust in A: 52.5 -> 50.0 (A reported network trouble on B ... that D does not see itself)” and then “Agent D marks A as SUSPECT: trust 49.9 fell below 50”.
- Middle: B, C and D: “is not voting to contain B: own observation of B normal (confidence 0.00 < 0.5)”.
- Then B, C and D, at almost the same moment: “will no longer count A's votes: trust 39.9 is below the vote-exclusion threshold 40”, and “is not counting A's vote to CONTAIN B”.
- Top: “Agent A re-sent its signed vote to contain B: [simulated compromise] fabricated accusation”. Interestingly, A's own honest checking still reports B as normal (the row “Agent A is not voting ... own observation of B normal”); only its compromised part votes.

See also Figure 7 (pending votes: stuck at 1/3), Figure 4 (trust trend) and Figure 5 ($W(B) = 0.012$) in Section 4.

Takeaway: One lying guard cannot get a healthy part locked, and it is caught and silenced automatically, without any human involved. Live: flagged in about 29–31 seconds, zero false isolations.

5.4 A forgery attempt: pretending to be another guard

What we do: guard A tries to impersonate guard B: it sends reports that *claim* to be from B, but it can only seal them with its own key and deliver them with its own badge. (In this mode A also sends its own fake accusations, as in 5.3.)

What happens: every guard compares the claimed sender with the badge of whoever delivered the message. They do not match, so the message is thrown away, and A loses 20 trust points **per forged message**.

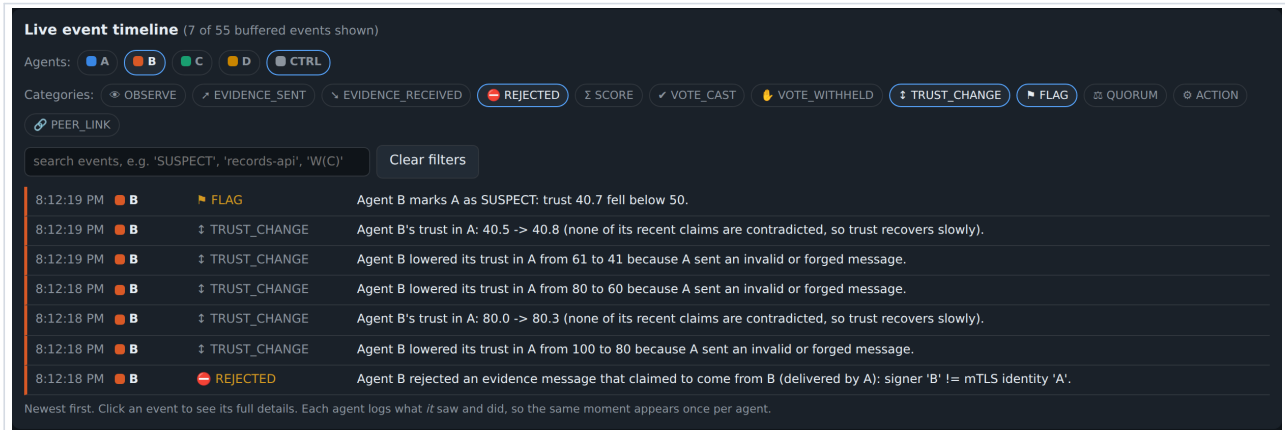


Figure 20. Guard B catching the forgeries (filtered to guard B's events).

What you're looking at

- Bottom row: **REJECTED**: "Agent B rejected an evidence message that claimed to come from B (delivered by A): signer 'B' != mTLS identity 'A'".
- Then the penalties, one per forged message: "lowered its trust in A from **100 to 80**", "**80 to 60**", "**61 to 41**". (The small "80.0 -> 80.3" rows between are the normal slow recovery.)
- Top: **FLAG**: "Agent B marks A as SUSPECT: trust 40.7 fell below 50", within about two seconds.
- Guard B even rejected a message pretending to come from B itself.

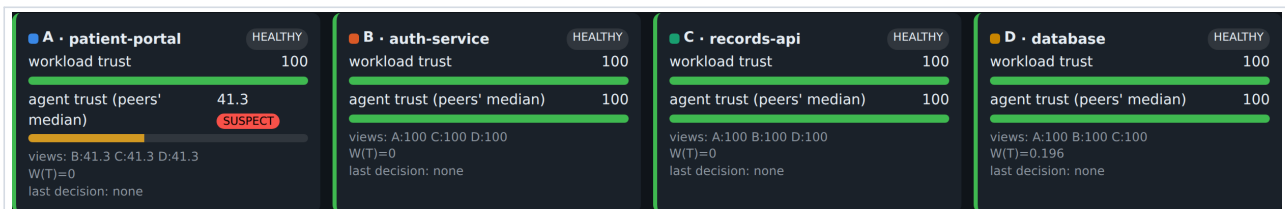


Figure 21. After the forgery attempt: A is SUSPECT, and nothing was locked.

What you're looking at

- A's card: agent trust **41.3**, SUSPECT. The views B:41.3 C:41.3 D:41.3 agree.
- D's card: W(T)=0.196: A's own (genuinely sealed) fake accusations about D, at a low weight. All four parts remain HEALTHY.
- The rejected-messages table for this run is Figure 10.

Takeaway: Signatures plus ID badges make impersonation impossible, and every attempt is punished instantly. Live: flagged in about 1.1 seconds; every forged message rejected.

5.5 A guard crashes, then a real attack happens

What we do: kill guard D completely (its program, its connections and its status page), then attack Node C exactly as in 5.2.

What happens: A, B and C notice they can no longer reach D. They carry on with three guards, which is still enough for a 3-of-4 decision, and complete the whole cycle without D.

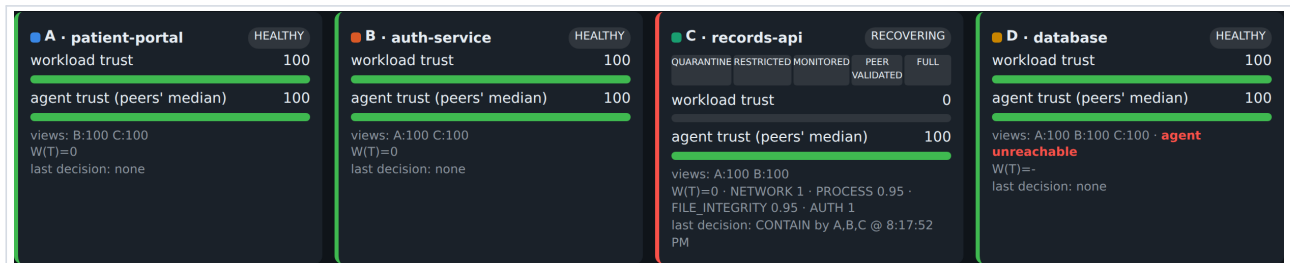


Figure 22. D's guard is down; the other three contain C on their own.

What you're looking at

- D's card: "views: A:100 B:100 C:100 · **agent unreachable**" in red. D's *database* is still HEALTHY; only its guard is gone.
- C's card: **RECOVERING**, "last decision: CONTAIN by **A,B,C**": exactly three signatures, without D.
- Every other card's views now list only the guards that are still running.

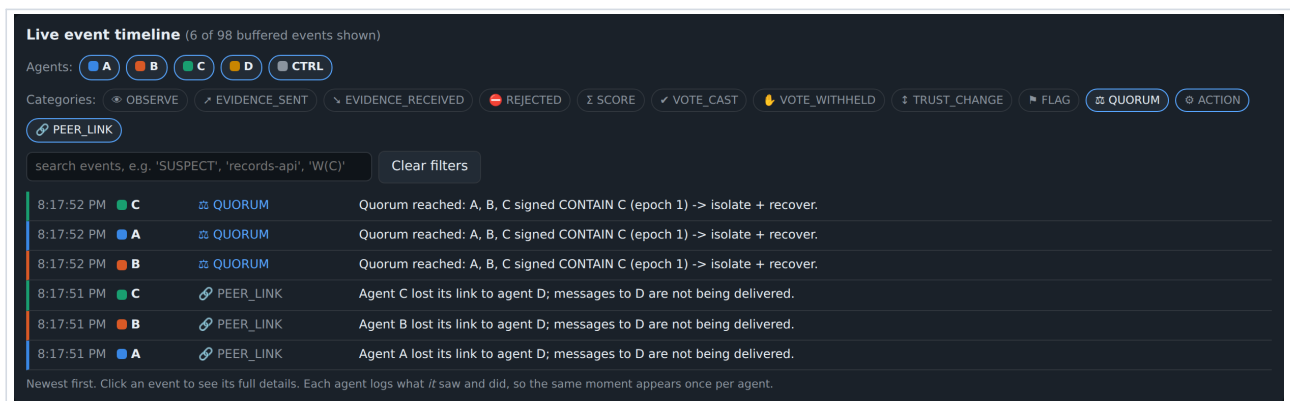


Figure 23. Losing contact with D, then deciding without it.

What you're looking at

- Bottom three rows: **PEER_LINK**: "Agent A/B/C lost its link to agent D; messages to D are not being delivered".
- One second later, top three rows: **QUORUM**: "Quorum reached: A, B, C signed CONTAIN C (epoch 1) -> isolate + recover", seen by each surviving guard.

Recent quorum decisions (click a row to see the evidence and votes that justified it)						
time	action	target	epoch	stage	signed by	
8:18:12 PM	ADVANCE_STAGE	C	1	FULL	A B C	► why?
8:18:09 PM	ADVANCE_STAGE	C	1	PEER_VALIDATED	A B C	► why?
8:18:05 PM	ADVANCE_STAGE	C	1	MONITORED	A B C	► why?
8:18:02 PM	ADVANCE_STAGE	C	1	RESTRICTED	A B C	► why?
8:17:58 PM	VALIDATE	C	1		A B C	► why?
8:17:52 PM	CONTAIN	C	1		A B C	► why?

Figure 24. The full recovery, every step signed by the three surviving guards.

What you're looking at

- CONTAIN, VALIDATE, then RESTRICTED, MONITORED, PEER_VALIDATED and FULL: every row signed by **A, B, C**.
- C went through the complete cycle with one guard permanently down.

Takeaway: Losing one guard does not stop the protection. The system degrades gracefully instead of failing.

5.6 The centralized baseline: one compromised controller

What we do: switch to the comparison version: **one** central controller, the traditional design, with the same detection and the same actions but no voting and no trust. Then we compromise the controller exactly as we compromised guard A in 5.3: it “accuses” the healthy login service B.

Distributed Cyber-Resilience Platform
LIVE
Pause
Export events

mode: centralized · data from 8:11:00 PM · centralized baseline running

A · patient-portal
HEALTHY

centralized mode: this node is watched only by the single central controller (no agents, no votes, no trust scores, no staged return: straight back to full access)

detections: last decision: none

B · auth-service
HEALTHY

centralized mode: this node is watched only by the single central controller (no agents, no votes, no trust scores, no staged return: straight back to full access)

detections: last decision: none

C · records-api
HEALTHY

centralized mode: this node is watched only by the single central controller (no agents, no votes, no trust scores, no staged return: straight back to full access)

detections: last decision: none

D · database
HEALTHY

centralized mode: this node is watched only by the single central controller (no agents, no votes, no trust scores, no staged return: straight back to full access)

detections: last decision: none

Figure 25. Centralized mode: one controller, no guards.

What you're looking at

- Header: **mode: centralized**.
- Each card says the part is “watched only by the single central controller (no agents, no votes, no trust scores, no staged return)”. Everything is healthy.

Distributed Cyber-Resilience Platform
LIVE
Pause
Export events

mode: centralized · data from 8:11:04 PM · centralized baseline running · controller compromise simulation: false-accusation

A · patient-portal
HEALTHY

centralized mode: this node is watched only by the single central controller (no agents, no votes, no trust scores, no staged return: straight back to full access)

detections: last decision: none

B · auth-service
RECOVERING

centralized mode: this node is watched only by the single central controller (no agents, no votes, no trust scores, no staged return: straight back to full access)

detections: last decision: CONTAIN QUARANTINE by CENTRAL @ 8:11:03 PM

C · records-api
HEALTHY

centralized mode: this node is watched only by the single central controller (no agents, no votes, no trust scores, no staged return: straight back to full access)

detections: last decision: none

D · database
HEALTHY

centralized mode: this node is watched only by the single central controller (no agents, no votes, no trust scores, no staged return: straight back to full access)

detections: last decision: none

Figure 26. One second later: healthy B is locked by the compromised controller.

What you're looking at

- Header: “controller compromise simulation: false-accusation”.
- B's card: **RECOVERING**, red border: the healthy login service has been cut off and is being rebuilt for no reason. “Last decision: CONTAIN QUARANTINE by **CENTRAL**”.
- In the real app, nobody can log in while B is locked, so the whole hospital portal stops working.

Live event timeline (2 of 6 buffered events shown)

Agents: A B C D CTRL

Categories: OBSERVE EVIDENCE_SENT EVIDENCE_RECEIVED REJECTED SCORE VOTE_CAST VOTE_WITHHELD TRUST_CHANGE FLAG QUORUM ACTION

PEER_LINK

search events, e.g. 'SUSPECT', 'records-api', 'W(C)' Clear filters

8:11:03 PM	CTRL	ACTION	The central controller isolated auth-service (B) and started redeploying it from the known-good image.
8:11:03 PM	CTRL	QUORUM	The central controller alone decided to CONTAIN B (compromised controller order); no second opinion is required or possible.

Newest first. Click an event to see its full details. Each agent logs what it saw and did, so the same moment appears once per agent.

Figure 27. The controller's own log: nobody else was asked.

What you're looking at

- **QUORUM**: “The central controller alone decided to CONTAIN B (compromised controller order); no second opinion is required or possible.”
- **ACTION**: “The central controller isolated auth-service (B) and started redeploying it”, in the same second.
- Compare with Figure 7, where the same accusation in our design is stuck at 1 of 3 votes.

Recent quorum decisions (click a row to see the evidence and votes that justified it)					
time	action	target	epoch	stage	signed by
8:11:03 PM	CONTAIN	B	1	QUARANTINE	CTRL

Figure 28. The decisions table: a single signer.

What you're looking at

- CONTAIN B, stage QUARANTINE, signed by **CTRL** only. One signature was enough; that is the weakness.

Takeaway: The same false accusation that our four guards ignore is carried out instantly by a single controller. Live: healthy auth-service falsely isolated within about 4 seconds, with the NetworkPolicy actually created.

5.7 The centralized controller crashes, then a real attack happens

What we do: stop the central controller, then attack Node C exactly as in 5.2, and wait 8 seconds (during 5.2 our guards had contained the attack in about one second).

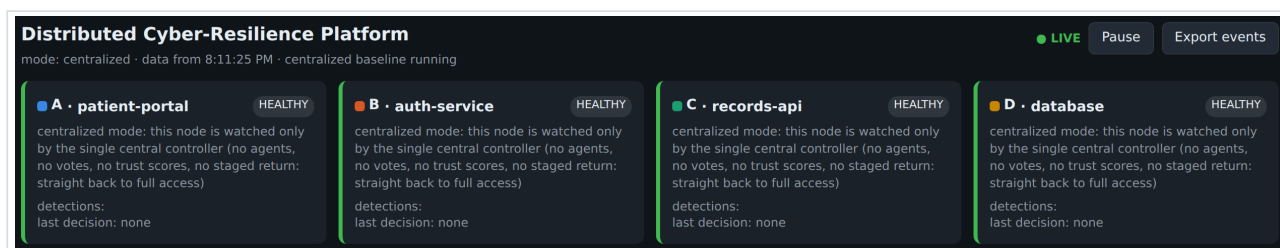


Figure 29. 8 seconds into a real attack on C, with the controller down.

What you're looking at

- All four cards still say **HEALTHY**, including C, which is actually under attack.
- Nothing is isolated, nothing is rebuilt: “last decision: none” everywhere.
- (The dashboard is showing the controller's last report from before it stopped.)

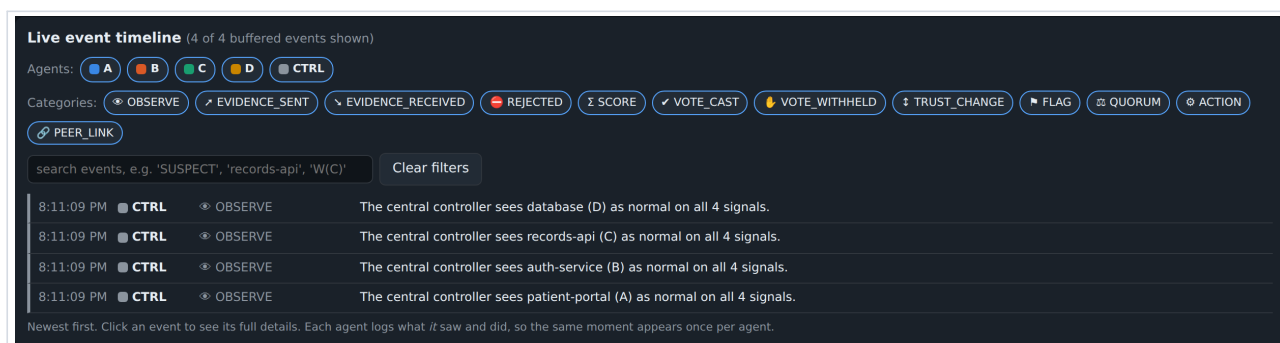


Figure 30. The controller's log stops at the moment it crashed.

What you're looking at

- The last four rows are its final routine check, all “normal on all 4 signals”, at 8:11:09.
- Then nothing: no observation of the attack, no decision, no action.

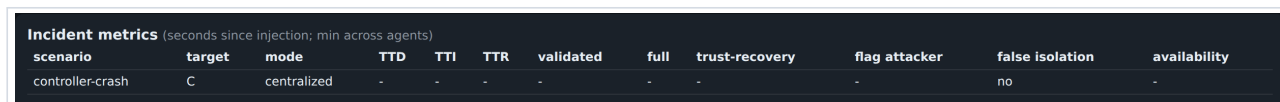


Figure 31. Metrics for the controller-crash incident: every time is blank.

What you're looking at

- TTD, TTI, TTR, validated, full: all “-”. The attack was never detected, isolated or recovered.
- Compare with Figure 22 and Figure 24 (agent crash), where losing one of four guards changed nothing.

Takeaway: With one controller, a crash means no protection at all. With four guards, losing one changes nothing.

6. Results

Two kinds of numbers are reported. **LIVE** numbers were measured on the real 4-node Minikube cluster (live timers: 1-second checks, 8-second grace, 10 seconds minimum per reintegration step). **SIMULATOR** numbers come from three full runs of `sim/local_demo.py`, which runs the real guard code with faster timers (half-second checks, 4-second grace, 3 seconds per step, a 2-second pretend rebuild); the simulator is therefore faster than a real cluster and its numbers show behaviour, not real-world speed.

Scenario	Where	Result
False accusation (guard A accuses healthy B)	LIVE	Attacker flagged SUSPECT in ~29-31 s ; no false isolation
Forgery (A impersonates B)	LIVE	Attacker flagged in ~1.1 s ; every forged message rejected
Centralized baseline, compromised controller	LIVE	Healthy auth-service falsely isolated within ~4 s (NetworkPolicy created)
Genuine compromise of C, full cycle	SIMULATOR (3 runs)	detect ≤ 0.5 s · isolate 1.0–1.5 s · recovered 3.0–4.0 s · validated 4.0–5.0 s · full access 18.0–19.0 s (of which ~13.5–14 s is the staged climb) · no false isolation
Agent D crashed, then genuine compromise of C	SIMULATOR (3 runs)	detect ≤ 0.5 s · isolate ~1.0 s · recovered ~3.0 s · validated ~4.0 s · full access ~17.5 s · every decision signed by A, B, C
Central controller crashed, then genuine compromise of C	SIMULATOR (3 runs)	no detection, no isolation, no recovery during the observed window; no NetworkPolicy created
For comparison: healthy centralized controller, genuine compromise	SIMULATOR	isolate ~0 s · full access ~2.5 s (faster, but no staged return and no protection if it is compromised or down)

- All 6 simulator scenarios passed in each of the three runs (18 of 18). The project's 57 automated tests also pass.
- Why the live false-accusation flag takes ~30 s: 8 s grace + ~20 s to fall from 100 to 50 at 2.5 points per second, plus measurement timing. It is a tunable setting, deliberately cautious so an honest guard that is a moment late is not punished.
- Why forgery is caught in ~1 s: a forged message costs 20 points at once, so a few forged messages cross the SUSPECT line immediately.
- The centralized controller *is* faster when everything is fine. That is the honest trade-off: we pay about a second of voting and a slower, careful return in exchange for surviving a broken or hacked security component.

7. A real bug we found and fixed

This is a good story to tell, because it shows why testing on the real system matters.

What went wrong

On the live cluster, the false-accusation test did nothing: guard A was switched to lying mode, but the other guards' trust in A never dropped, and the logs showed nothing at all. Yet in the simulator the same test worked perfectly.

The cause, in plain English

Each guard runs a small “post office” (a **gRPC server**: gRPC is the messaging system the guards use to talk to each other) that receives the other guards' letters. In the live start-up code, the post office was opened, but the program did not keep hold of it: nothing in the program “remembered” it afterwards.

Python, the language the guards are written in, has a **garbage collector**: an automatic cleaner that throws away anything the program is no longer holding on to. About one second after start-up, the cleaner threw the post office away, and the gRPC library closes a server when that happens. From then on every guard silently refused all incoming letters. A's lies were sent, but nobody ever received them.

Everyday analogy. You open a shop, but forget to keep the keys. The cleaner sees an unattended shop and locks it up. Customers (letters) arrive and find the door shut, and because nobody is inside, nobody notices.

Why the tests and the simulator missed it

The simulator starts the same post offices, but it keeps them in a list, so they are always “held” and never cleaned away. The automated tests used the simulator's start-up path. The only code with the bug was the live start-up path, which the tests did not run. On top of that, failed deliveries were logged only at the most hidden log level, so the failure was silent.

How we found it

- We rebuilt the exact live environment in Docker (the real guard program image, the same network names, the same certificates) and reproduced the failure: every delivery failed with “connection refused”.
- We checked which network ports were open inside a guard: the post office port (50051) was closed.
- We then proved the cause directly: a post office that the program keeps hold of stays open; one that it does not keep is closed about one second later.

The fix

- The live start-up code now keeps hold of the post office for the life of the guard.
- A lost connection to another guard is now reported loudly (“peer link to D DOWN”), and shown on the dashboard as a PEER_LINK event, so this kind of failure can never be silent again.

- New automated tests run the *live* start-up path and force the cleaner to run, then check that every guard can still reach every other guard.

Takeaway: Simulators and tests can share blind spots with the code they test. Rebuilding the real environment exactly is what exposed this bug.

8. Honest limitations and future work

Saying clearly what the project does *not* do is part of doing it properly.

Limitation	What it means	Future work
No live application-level attack injection	On the live cluster we cannot make a hospital part really misbehave on demand, so the genuine-compromise scenario (5.2) is demonstrated with the simulator. The false-accusation, forgery and baseline tests <i>are</i> live.	A safe, controlled way of triggering real misbehaviour inside the live cluster.
Simple threshold detection	Each signal is compared with a fixed limit. It is transparent and easy to explain, but can miss clever attacks or raise false alarms.	Machine-learning detection, e.g. Isolation Forest (a method that learns what “normal” looks like and flags unusual behaviour).
One guard could technically act alone	Each guard has permission to change network rules. Our guards only act with 3 signed votes, but a fully hacked guard could bypass its own code.	An admission check in Kubernetes (a gatekeeper that inspects every change) that rejects any isolation without a valid 3-vote certificate.
Healthcare is only the demo	The hospital app, with randomly generated fake patients, is just an example. The platform itself works with any containerised cloud app.	Test with other kinds of applications.
Network partition not tested	If the network splits the guards into two groups of two, neither group can reach 3 votes, so the system safely does nothing, but it also cannot act.	Measure and handle partitions.
Only one bad guard is tolerated	With 4 guards the maths tolerates exactly 1 bad one. We have not yet run the 2-of-4-compromised experiment that would show the breaking point.	Run it and report the limit honestly; use 7 guards to tolerate 2.
Other misbehaviour of a hacked guard	We test a guard that lies and one that forges. A guard that goes silent, or tries to block a correct lockdown, is not yet tested as an attack scenario.	Add those scenarios.
Isolation before investigation	The system isolates first; it does not investigate or ask a human first.	An investigation step or human approval for serious cases.
No real medical data	All patient data is randomly generated.	Use realistic datasets such as Synthea or MIMIC.

9. What we do NOT claim

Following section 12 of our project specification, we are careful not to overclaim:

- **Not** “the first distributed cybersecurity system”. That would be false; many exist.
- **Not** “quorum voting is our invention”. It is a well-established technique from computer science (Byzantine fault tolerance, PBFT by Castro and Liskov).
- **Not** “our system always identifies the correct malicious node”. No system can guarantee that if enough participants are compromised; with 4 guards we tolerate exactly one.
- **Not** “nothing like this exists”. Instead we name the closest prior work and say exactly how we differ.

Closest prior work, and our difference, in one line each:

Existing work	What it does	How we differ
Dempster-Shafer evidence fusion in intrusion detection (2000s)	combines alerts from several sensors, weighted by confidence	we use weighted evidence specifically to authorise containment of containers
PBFT / Castro-Liskov proactive recovery	formal agreement protocol plus periodic recovery	we use the voting idea as a building block and add trust-scored, staged reintegration
TOLERANCE (2025)	automatic recovery on top of agreement protocols	we test in a cloud-container setting with continuous trust scores instead of a yes/no recovery
Distributed trust frameworks for intrusion detection (2011)	trust scores to detect compromised hosts	we use trust to control <i>how fast</i> a part is re-admitted, and test with a lying guard

What we do claim (say this sentence): “We build on established Byzantine fault-tolerant voting and evidence-fusion techniques. Our contribution is (1) a continuous trust-decay and recovery model that governs step-by-step reintegration, and (2) an evaluation of the full detect-to-reintegrate cycle when the security system itself is the target, in a containerised cloud application.”

10. Likely faculty questions, with short answers

Q: How is this different from SOAR tools?

SOAR (Security Orchestration, Automation and Response) tools, such as those in big companies, run automated response playbooks from one central server. That server is exactly the single point of failure and compromise we remove. Ours has no central server: four guards must agree, and a lying guard is caught automatically.

Q: What if the detector is wrong?

A single guard's mistake cannot cause a lockdown: three guards must each see the problem themselves. If all guards are wrong together (a bad threshold), we could isolate a healthy part, but it is then rebuilt, validated and re-admitted automatically, and the decision's evidence is recorded for review. Better detection (e.g. machine learning) is future work.

Q: What if 2 of the 4 guards are compromised?

Then the guarantee no longer holds: the maths ($n = 3f + 1$) says 4 guards tolerate exactly 1 bad one. Two liars could together push a score over 0.6, although they still need a third, honest vote to act. We say this openly, and running this breaking-point test is future work. To tolerate 2, use 7 guards.

Q: Why 4 agents?

Four is the smallest number that can survive one dishonest member, with decisions needing 3 votes. Three guards could not safely tell a liar from an honest guard.

Q: Why not use machine learning?

Detection was deliberately kept simple and explainable, because the contribution is what happens after detection: agreement, trust and safe recovery. Any detector, including ML such as Isolation Forest, can be plugged in later and would benefit from the same voting and trust protection.

Q: Is this really cloud-ready?

It runs on Kubernetes, the standard platform used by real cloud providers, and uses only standard building blocks (NetworkPolicies, deployments, certificates). It was tested on a real 4-node Minikube cluster. It is a working prototype, not a hardened product: Section 8 lists what production would need.

Q: What did you build and what did you reuse?

Built: the guard program (monitoring, detection, signed evidence, the scoring, trust, voting and the whole isolate-recover-validate-reintegrate pipeline), the message protocol, the centralized baseline for comparison, the dashboard, the simulator, the deployment scripts and the tests. **Reused:** Kubernetes, Minikube and Calico, the gRPC messaging library, standard cryptography libraries and the Ed25519 signature method, and the established theory of Byzantine fault tolerance.

Q: Doesn't voting make it slower?

A little: contain took about 1 second, versus about 0 for a single controller, and the careful staged return takes longer. That is the price of not trusting any single component.

Q: How do you tell a lying guard from an honest one that is simply wrong?

We do not need to. Either way its claims are not confirmed by the others, so its trust falls and its influence shrinks. If it was only temporarily wrong, its trust recovers slowly once it behaves.

Q: Could the attacker just attack the dashboard?

The dashboard only displays information; it has no power to make decisions or change anything. Breaking it would hide the view, not the protection.

Q: Why not use a blockchain?

We only need four known guards to agree, quickly. A blockchain is designed for thousands of unknown participants and would add delay and complexity without extra safety here.

Q: What happens if the guards cannot reach each other at all?

Then no group reaches 3 votes, so nothing is locked: the safe default. The trade-off is that nothing can be locked either; handling network splits is future work.

11. Demo cheat sheet

Run from the project folder, with the Python environment active (source .venv/bin/activate). Keep the dashboard open in the browser at <http://localhost:8090> the whole time.

#	Command	What to say / show
0	<code>kubectl config use-context cr-platform scripts/deploy.sh # only if not already running kubectl -n resilience port-forward svc/dashboard 8090:8090 &</code>	"Four hospital parts, four guards, all green. No boss."
1	(point at the dashboard)	Explain a card: letter, state, workload trust, agent trust. Click card B to show its own view.
2	<code>scripts/simulate-compromised-agent.sh A B # wait ~30 s, then: scripts/simulate-compromised-agent.sh A restore</code>	"A is now a lying insider accusing healthy B. Watch Pending votes: 1 of 3, and the others say 'own observation of B normal'. Watch A's trust fall, SUSPECT, votes ignored. B is never locked."
3	<code>scripts/simulate-compromised-agent.sh A C forge-evidence scripts/simulate-compromised-agent.sh A restore</code>	"Now A pretends to be B. Rejected messages: 'signer B, badge A'. Minus 20 per forgery."
4	<code>scripts/reset.sh scripts/switch-mode.sh baseline scripts/simulate-compromised-controller.sh B kubectl -n healthcare get networkpolicy</code>	"Same lie, one boss: healthy login service locked in seconds, signed by CTRL alone."
5	<code>scripts/simulate-compromised-controller.sh restore scripts/reset.sh scripts/switch-mode.sh distributed</code>	"Back to our design."
6	<code># second terminal: python3 sim/local_demo.py app-compromise --serve # third terminal: STATUS_URLS=http://localhost:50251/status,http:// localhost:50252/status,\ http://localhost:50253/status,http://localhost:5 0254/status \ PORT=8091 python3 dashboard/server.py # open http://localhost:8091</code>	"The full cycle on a real attack, shown in the simulator: detect, vote, isolate, rebuild, validate, five steps back." Press Pause to explain a step.
7	<code>python3 sim/local_demo.py agent-crash --serve # same dashboard command</code>	"Kill guard D: the other three still reach 3 votes and finish."
8	<code>python3 sim/local_demo.py controller-crash --serve STATUS_URLS=http://localhost:50261/status BASELINE_URL=http://localhost:50261 \ PORT=8091 python3 dashboard/server.py</code>	"Kill the single controller: the attack is simply never seen."
9	<code>python3 scripts/collect-metrics.py --url http://localhost:8090 # Export events button = report data</code>	Show the times. Close with the 'what we claim' sentence from Section 9.

If the live demo misbehaves: run `python3 sim/local_demo.py` (all 6 scenarios, about 2 minutes, ends with 6 × PASS) and use the screenshots in Section 5.

12. Glossary

Term	Meaning
Admission check	A Kubernetes gatekeeper that inspects every requested change and can refuse it.
Agent (resilience agent)	One of our four guard programs; it watches the apps, reports, votes and acts.
Agent trust	How much the other guards believe a given guard (0–100); shown as the median of their views.
Availability	Whether the application keeps working for its users during an attack.
Baseline (centralized)	Our comparison version with one central controller and no voting.
Byzantine fault tolerance (BFT)	Designing a group so it still decides correctly even if some members lie.
Calico	The networking add-on for Kubernetes that actually enforces NetworkPolicy rules.
Certificate / certificate authority	A digital ID badge, and the trusted office that issues the badges.
Cluster	A group of computers managed together by Kubernetes.
Confidence	A guard's certainty, from 0 to 1, that something is wrong.
Container	A sealed, portable box holding one piece of software and everything it needs to run.
Contain	Our word for the decision to isolate a part and rebuild it.
Dashboard	The read-only web page that shows what the guards are doing.
Diversity bonus	Extra score (0.15 each) when different kinds of evidence confirm the same problem.
Ed25519	The modern digital-signature method the guards use to seal their messages.
Epoch	The incident number for a given part (epoch 1 = its first incident).
Evidence	A signed report from one guard saying what it saw on which part, how sure it is, and when.
Executor	The guard that carries out an agreed action; the others step in only if it does not.
False isolation	Wrongly cutting off a healthy part; the main thing a lying guard tries to cause.
Garbage collector	Python's automatic cleaner that throws away anything the program is no longer holding.
gRPC	The messaging system the guards use to send each other messages over the network.
Hash (file fingerprint)	A short code calculated from a file's contents; any change to the file changes it.

Term	Meaning
Image (known-good image)	A sealed master copy of the software, used to rebuild a clean part.
Isolation Forest	A machine-learning method that learns normal behaviour and flags unusual behaviour.
Kubernetes	The standard system for running, connecting and managing containers.
Minikube	A tool that runs a small Kubernetes cluster on one computer, for testing.
mTLS (mutual TLS)	Both sides of a connection prove who they are with certificates, and the traffic is encrypted.
Network partition	When the network splits, so some computers cannot reach others.
NetworkPolicy	A Kubernetes firewall rule saying which parts may talk to which.
Node	One computer in the cluster; in our demo each node hosts one hospital part and one guard.
PBFT	Practical Byzantine Fault Tolerance, a classic voting protocol by Castro and Liskov.
Pod	Kubernetes' smallest unit: one or more containers running together.
Quorum / quorum certificate	The minimum number of votes needed (3 of 4), and the bundle of those signed votes.
Reintegration	Letting a repaired part back in, one step at a time.
Replay attack	Re-sending an old, genuine message to trick the receiver; blocked by freshness checks.
SIMULATOR / LIVE	Results from our one-machine simulator (faster timers) versus the real cluster.
SOAR	Security Orchestration, Automation and Response: tools that run automated responses from a central server.
Signature (digital)	A mathematical seal only the key-holder can make but anyone can check.
Single point of failure / compromise	One part whose failure (or hacking) stops or hijacks the whole system.
Support	Whether enough guards reported a signal type: one guard alone gets half weight.
SUSPECT	The flag shown when a guard's trust falls below 50.
Synthea / MIMIC	Realistic medical datasets (synthetic and real, anonymised) for future testing.
Threshold / limit	The value above which a signal counts as suspicious.
Timeline (event log)	The dashboard list of every guard's actions and reasons, one sentence each.
TTD / TTI / TTR	Time to detect / time to isolate / time to recover, measured from the start of an attack.
Vote-exclusion threshold	Trust level (40) below which a guard's votes are ignored.

Term	Meaning
W(T), weighted score	The combined, trust-weighted evidence score about part T; 0.6 is needed to vote.
Workload	One part of the application being protected, e.g. records-api.
Workload trust	How healthy the guards believe a part is (0-100); it controls the reintegration steps.